

WEEK-12

Task-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

```
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int BUFFER_SIZE;

int in = 0, out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < ITEMS; i++) {

        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg) {
    long sum = 0;

    for (int i = 0; i < ITEMS; i++) {

        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        sum += buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
    }
}
```

OUTPUT:

```
Expected Counter = 400000
pavani@DESKTOP-380LEIN:~/practical$ nano producer_consumer.c
pavani@DESKTOP-380LEIN:~/practical$ gcc producer_consumer.c -o producer_consumer -pthread
pavani@DESKTOP-380LEIN:~/practical$ ./producer_consumer 1
Consumer sum = 4999950000
Buffer size = 1
Items processed = 100000
Time = 4.011345 seconds
Throughput = 24929.29 items/sec
pavani@DESKTOP-380LEIN:~/practical$ ./producer_consumer 10
Consumer sum = 4999950000
Buffer size = 10
Items processed = 100000
Time = 0.367406 seconds
Throughput = 272178.44 items/sec
pavani@DESKTOP-380LEIN:~/practical$
```

Report:

The Producer-Consumer problem was implemented using POSIX threads, counting semaphores, and a mutex.

The empty and full semaphores controlled the available and occupied buffer slots.

Different buffer sizes were tested and execution time and throughput were measured.

The experiment demonstrated correct synchronization and showed how buffer capacity can affect waiting and throughput.

Task-2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

pavani@DESKTOP-38OLEIN: ~/practical

GNU nano 8.7.1

deadlock.c *

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg) {
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg) {
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
```

OUTPUT:

```
pavani@DESKTOP-380LEIN:~/practical$ gcc deadlock.c -o deadlock -pthread
pavani@DESKTOP-380LEIN:~/practical$ ./deadlock
Thread 2: locking Resource 2
Thread 1: locking Resource 1
Thread 1: waiting for Resource 2
Thread 2: waiting for Resource 1
^C
pavani@DESKTOP-380LEIN:~/practical$
```

```
GNU nano 8.7.1 no_deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg) {
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg) {
    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
```

```
pavani@DESKTOP-380LEIN:~/practical$ nano no_deadlock.c
pavani@DESKTOP-380LEIN:~/practical$ gcc no_deadlock.c -o no_deadlock -pthread
pavani@DESKTOP-380LEIN:~/practical$ ./no_deadlock
Thread 1: locking Resource 1
Thread 2: locking Resource 1
Thread 1: locking Resource 2
Thread 1: acquired both resources
Thread 2: locking Resource 2
Thread 2: acquired both resources
Program completed without deadlock.
pavani@DESKTOP-380LEIN:~/practical$
```

Report: A deadlock scenario was created using two threads and two shared mutex resources. The program demonstrated mutual exclusion, hold-and-wait, no preemption, and circular wait. Deadlock was prevented by requiring all threads to acquire resources in the same predefined order.

The synchronized version completed successfully without entering a deadlock state.